国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「 Course 」
[Course name]

「 Lesson 3 」 EDHOC (RFC 9528) on two boards

[Author]

[YYYY/MM]

Today's lesson purpose:

- Understand EDHOC: three messages that authenticate two devices and give them a shared secret (RFC 9528).
- Know the four authentication methods, the nine cipher suites and the key schedule, and read the messages byte by byte.
- Build, run and measure all 36 method × suite combinations on two boards over the radio, and check the code against the RFC 9529 test vectors.

Code: fpga/sw/export/edhoc-wolfssl · standard: RFC 9528 (2024), examples: RFC 9529

Outline

1. EDHOC in a Nutshell
2. Run One Handshake
3. Key Schedule
4. Inside the Messages
5. The Library
6. What to Expect
7. Practice

Outline

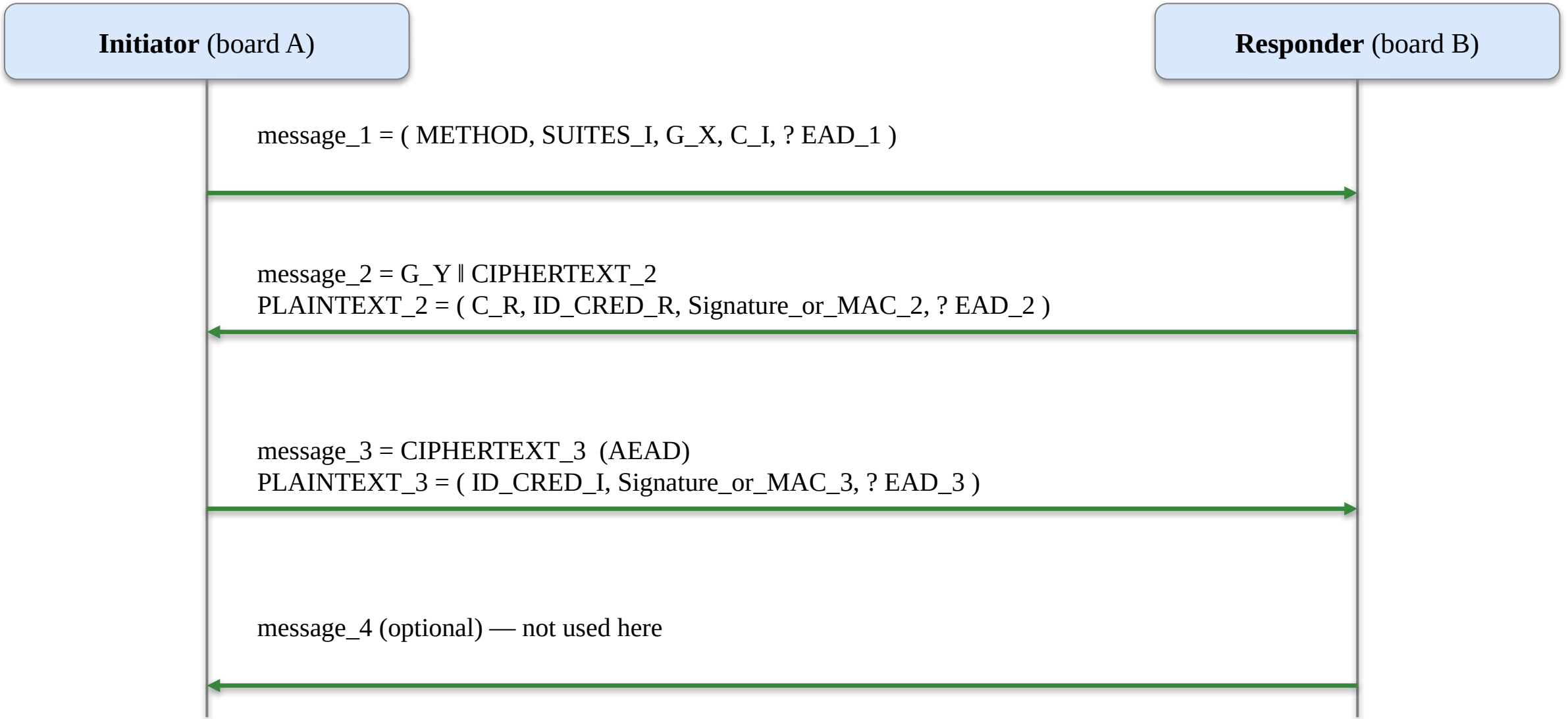
1. **EDHOC in a Nutshell**
2. Run One Handshake
3. Key Schedule
4. Inside the Messages
5. The Library
6. What to Expect
7. Practice

1. EDHOC in a Nutshell (1/5) What EDHOC is

- **E**phemeral **D**iffie-**H**ellman **O**ver **C**OSE: an authenticated key exchange for small devices and slow links (IETF LAKE, RFC 9528, March 2024).
- Three messages. After them both sides know who the other is (signature or static DH) and share a fresh secret **PRK_out** (forward secrecy).
- Messages are compact CBOR; credentials and keys use COSE — the formats OSCORE, CoAP and constrained IoT stacks already use.
- From PRK_out an application derives its own keys (*EDHOC_Exporter*): OSCORE in the standard, our encrypted file channel in Lesson 5.

Role	Here
Initiator	board A — starts the handshake
Responder	board B — waits for message_1

1. EDHOC in a Nutshell (2/5) Message flow



G_X, G_Y: ephemeral public keys · C_I, C_R: connection IDs · EAD: optional extra data · ‖ = concatenation

1. EDHOC in a Nutshell (3/5) Four authentication methods

METHOD	Initiator proves itself by	Responder proves itself by	Typical use
0	signature	signature	certificates, PKI
1	signature	static Diffie-Hellman key	constrained responder
2	static Diffie-Hellman key	signature	constrained initiator
3	static Diffie-Hellman key	static Diffie-Hellman key	smallest, fastest

- Signature side: *Signature_or_MAC* is a signature with the long-term key.
- Static-DH side: it is a short MAC, keyed by a DH between the peer's ephemeral key and this side's static key — no signature algorithm needed.
- All four run on the boards (36 combinations with the 9 suites).

1. EDHOC in a Nutshell (4/5) Nine cipher suites

Suite	EDHOC AEAD	Hash	Key exchange	Signature	App AEAD
0	AES-CCM-16-64-128	SHA-256	X25519	EdDSA	AES-CCM-16-64-128
1	AES-CCM-16-128-128	SHA-256	X25519	EdDSA	AES-CCM-16-64-128
2	AES-CCM-16-64-128	SHA-256	P-256	ES256	AES-CCM-16-64-128
3	AES-CCM-16-128-128	SHA-256	P-256	ES256	AES-CCM-16-64-128
4	ChaCha20/Poly1305	SHA-256	X25519	EdDSA	ChaCha20/Poly1305
5	ChaCha20/Poly1305	SHA-256	P-256	ES256	ChaCha20/Poly1305
6	AES-128-GCM	SHA-256	X25519	ES256	AES-128-GCM
24	AES-256-GCM	SHA-384	P-384	ES384	AES-256-GCM
25	ChaCha20/Poly1305	SHAKE256	X448	Ed448	ChaCha20/Poly1305

Every suite of the RFC 9528 registry. Default build: suite 2. Suite 24 is the strongest classical suite; post-quantum suites: Lesson 4.

1. EDHOC in a Nutshell (5/5) Credentials and ID_CRED

- **CRED_x**: the long-term public key, here as a CWT Claims Set (CCS) or an X.509 certificate hash (x5t).
- **ID_CRED_x**: a short reference to it sent in the message, e.g. `{4: h'02'}` = “key id 0x02”. The receiver looks the credential up in its trust anchors.
- RFC 9528 §3.5 requires the application to see ID_CRED before it is trusted. Our library calls an optional `id_cred_check()` hook; returning an error stops the handshake.
- Compact encoding: a one-byte kid travels as a single CBOR byte (e.g. `0x32`); the receiver turns it back into `{4: h'32'}`.

edhoc-wolfssl/src/edhoc_app.c

```
p->id_cred[0] = 0xa1;   /* map, 1 entry          */
p->id_cred[1] = 0x04;   /* label 4 = kid         */
p->id_cred[2] = 0x41;   /* byte string, 1 byte   */
p->id_cred[3] = kid;
```

Outline

1. EDHOC in a Nutshell
2. **Run One Handshake**
3. Key Schedule
4. Inside the Messages
5. The Library
6. What to Expect
7. Practice

2. Run One Handshake (1/7) Lab: one handshake, suite 2, method 3

Objective	authenticate board A and board B to each other and agree on PRK_out, then exchange encrypted data
Starting state	both FPGAs programmed (Lesson 1); boards.env filled in and both radios on JC (Lesson 2)
This example	method 3: both sides prove themselves with a static DH key; suite 2: P-256, SHA-256, AES-CCM-16-64-128
Credentials	demo keys in include/edhoc_demo_keys.h: each board has its own key pair and trusts the other's CCS (kid 0x01 / 0x02)

Run this

```
cd fpga/sw/export/edhoc-wolfssl
make edhoc_m3_initiator edhoc_m3_responder CRYPTO_SUITE=2 TIMING=1
VERBOSE=1 ../common/board/run_pair.sh s2_m3 build/edhoc_m3_initiator.elf \
                                            build/edhoc_m3_responder.elf
```

Example output — last line

```
s2_m3 PASS hs=2.740934s stack_i=5360 stack_r=4736 prk=- 7s
```

- Both consoles scroll live (I| initiator, R| responder) and are saved in *boardlogs/s2_m3.ini.log* and *.res.log*. The pair stops at the first FAIL or after 300 s (4th argument).

2. Run One Handshake (2/7) Initiator console (board A)

```
START (initiator, method 3, suite 2)
PERF keygen            64432101 cyc 0.859094 s
EDHOC running...
EDHOC OK!
PERF handshake_total   206397933 cyc 2.751972 s
PERF handshake_link    103804007 cyc 1.384053 s
PERF handshake_compute 102593926 cyc 1.367919 s
PERF msg1_build        38407 cyc 0.000512 s
PERF msg2_process      101441953 cyc 1.352559 s
PERF msg3_build        773221 cyc 0.010309 s
STACK peak 5376 B, reserve 8448 B, free 25856 B, headroom 20480
APP: channel up (aead=10 key=16 iv=13 tag=8)
APP: sent    'REQ 0 FROM INITIATOR'
APP: got     'ECHO[REQ 0 FROM INITIATOR]'
...
PERF encrypt      54759 cyc 0.000730 s 60 bytes 82191 B/s
APP DATA OK
```

- *keygen*: ephemeral key, before the handshake.
- *msg2_process* is the big step: decrypt, two P-256 DH operations (method 3), MAC check.
- *aead=10*: COSE algorithm 10 = AES-CCM-16-64-128, the suite's application AEAD.

2. Run One Handshake (3/7) Responder console (board B)

```
START (responder, method 3, suite 2)
PERF keygen            64138947 cyc 0.855185 s
EDHOC running...
EDHOC OK!
PERF handshake_total   295817506 cyc 3.944233 s
PERF handshake_link    194511405 cyc 2.593485 s
PERF handshake_compute 101306101 cyc 1.350748 s
PERF msg2_build        66894935 cyc 0.891932 s
PERF msg3_process      34056432 cyc 0.454085 s
STACK peak 4736 B, reserve 8448 B, free 25584 B, headroom 20848
APP: got     'REQ 0 FROM INITIATOR'
APP: sent    'ECHO[REQ 0 FROM INITIATOR]'
...
APP DATA OK
```

- The responder's total is longer: it starts waiting before the initiator is loaded. Compare **compute** times, not totals.
- Both computes ≈ 1.35 s: each side does two P-256 operations.
- The echo shows both sides derived compatible application keys: every record's AEAD tag verified. It is not a byte comparison of PRK_out (next slide).

2. Run One Handshake (4/7) What each result proves

You see	You may conclude	You may not conclude
EDHOC OK! (one console)	that board finished its side of the handshake	that the peer finished, or agrees
APP DATA OK (both)	every encrypted record decrypted with a valid tag, both directions → compatible application keys	that the data was stored anywhere
prk=match	run_pair.sh found PRK_out printed on both consoles and the bytes are equal	—
prk=-	no PRK_out was printed (release builds of edhoc-wolfssl): no direct comparison	that they differ
RFC 9529 test PASS	for the published inputs, every value equals the RFC	anything about other suites or methods
host loopback PASS	both roles of the same code agree on PRK_out	radio or board behaviour

A PASS line of run_pair.sh = EDHOC OK and APP DATA OK on both sides, no stack overrun, and prk not DIFFER.

2. Run One Handshake (5/7) If the pair fails

run_pair.sh says	Look at	Usual cause
LOADFAIL	boardlogs/<name>.load.log	cable serial in boards.env, JD wiring, FPGA not programmed
no-EDHOC-OK on both	both logs; rebuild with DEBUG=1 to see each message sent ([I-TX] / [R-TX])	radio not on JC, the two images built for different suites
EDHOC FAIL: <n> on one side	the error number; the other side got an error message	wrong credential, refused ID_CRED, mismatched method
no-APP-OK	the APP: lines	link lost after the handshake (Lesson 5, Problem 3)
OVERRAN(…)	the STACK line	reserve too small for this build: raise STACK_SIZE

- Rebuild both roles after any change: a pair of images from different flags does not interoperate.
- Run the same combination on the PC first ((7/7)): if the host test fails, the boards cannot pass.

2. Run One Handshake (6/7) Run and judge pairs automatically

```
Run this — from fpga/sw/export/edhoc-wolfssl, about 1 h
for s in 0 1 2 3 4 5 6 24 25; do for m in 0 1 2 3; do
  make -s edhoc_m${m}_initiator edhoc_m${m}_responder CRYPTO_SUITE=$s TIMING=1
  ../common/board/run_pair.sh s${s}_m$m build/edhoc_m${m}_initiator.elf \
                                        build/edhoc_m${m}_responder.elf
done; done
../common/board/perf_table.py boardlogs > timing.md
```

- The loop runs every method × suite pair unattended; one line per pair in *boardlogs/results.txt*, both consoles per pair in *boardlogs/*.
- *perf_table.py* turns the saved logs into the timing tables of section 6.
- Add `DEBUG=1` to the make line to print every intermediate value — for understanding, not for timing (section 6, (1/7)).

2. Run One Handshake (7/7) The same handshake on the PC

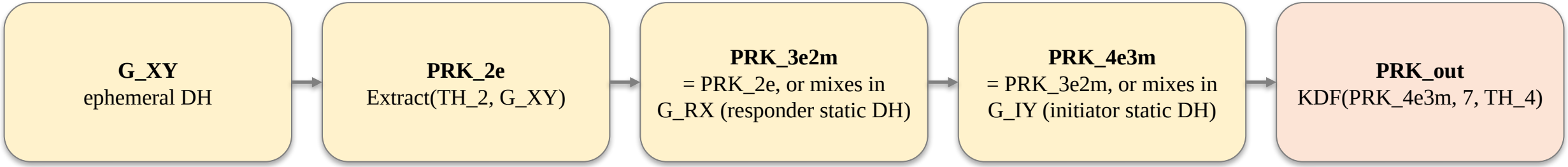
```
Run this — and the line it prints
cd fpga/sw/export/edhoc-wolfssl
make loopback-test CRYPTO_SUITE=24 METHOD=0
loopback suite 24 method 0: initiator 0 responder 0, PRK_out match -> PASS
```

- Initiator and responder as two threads, messages through memory.
- Same EDHOC and crypto code as the board, PRK_out of both sides compared.
- Expect PASS for every suite × method — run it after every library change, before touching the boards.

Outline

1. EDHOC in a Nutshell
2. Run One Handshake
3. **Key Schedule**
4. Inside the Messages
5. The Library
6. What to Expect
7. Practice

3. Key Schedule (1/3) From Diffie-Hellman to PRK_out



Transcript hash	Covers
TH_2 = H(G_Y, H(message_1))	message_1 and the responder's key share
TH_3 = H(TH_2, PLAINTEXT_2, CRED_R)	everything up to message_2 + responder credential
TH_4 = H(TH_3, PLAINTEXT_3, CRED_I)	everything + initiator credential

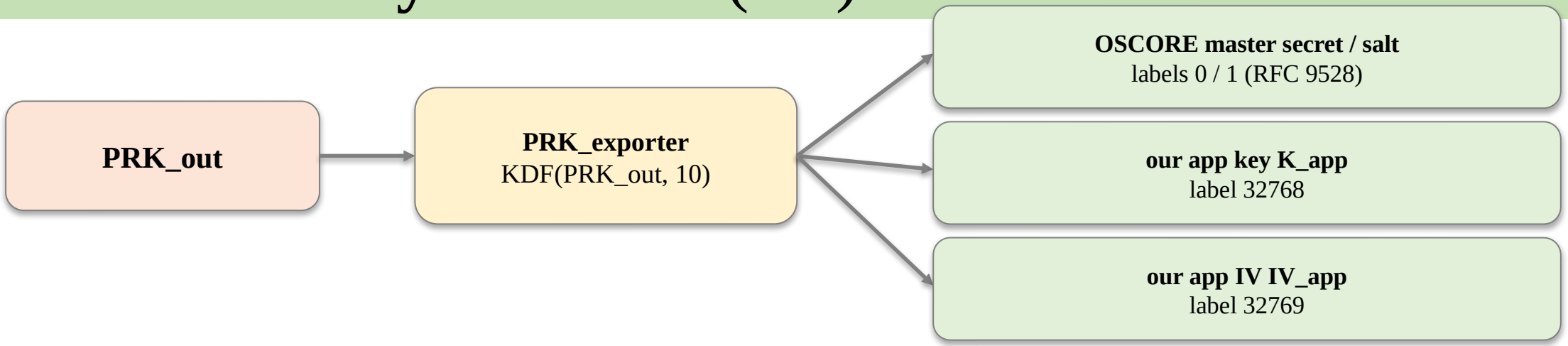
- Each key is bound to the whole conversation so far: changing any byte of any message changes every later key.
- Extract / KDF = HKDF with the suite's hash (SHA-256, SHA-384), or KMAC256 for the SHAKE256 suite 25.

3. Key Schedule (2/3) What each message protects

Message	Protection	Key
message_1	none (public key share, suites)	–
message_2	PLAINTEXT_2 XOR KEYSTREAM_2; MAC_2 inside	KEYSTREAM_2 from PRK_2e; MAC_2 from PRK_3e2m
message_3	AEAD (suite's EDHOC AEAD); MAC_3 inside	K_3 / IV_3 from PRK_3e2m; MAC_3 from PRK_4e3m

- Identities (ID_CRED) are encrypted: an eavesdropper on the radio does not learn who is talking.
- Every key comes from **EDHOC_KDF(PRK, label, context, length)** with a fixed label number: KEYSTREAM_2 = 0, SALT_3e2m = 1, MAC_2 = 2, K_3 = 3, IV_3 = 4, SALT_4e3m = 5, MAC_3 = 6, PRK_out = 7, PRK_exporter = 10.
- These are exactly the values the RFC 9529 test (section 3) compares byte by byte.

3. Key Schedule (3/3) After the handshake



- *EDHOC_Exporter(label, length)* gives each application its own key without re-running the handshake.
- Our demo first echoes three short messages (REQ/ECHO) both ways, then can stream a file (Lesson 5) with K_app / IV_app.
- PRK_out is wiped from memory once the application has its keys.

Outline

1. EDHOC in a Nutshell
2. Run One Handshake
3. Key Schedule
4. **Inside the Messages**
5. The Library
6. What to Expect
7. Practice

4. Inside the Messages (1/8) CBOR in five minutes

- EDHOC messages are **CBOR sequences**: items written back to back, no outer array. The first byte of an item gives its type (top 3 bits) and its value or length (low 5 bits).

Bytes	Major type	Meaning
03	0: unsigned integer	3
37	1: negative integer	−1 − 23 = −24
18 20	0: unsigned, value in the next byte	32
82 06 02	4: array of 2 items	[6, 2]
41 32	2: byte string of 1 byte	h'32'
58 20 …	2: byte string, length in the next byte	32 bytes follow
59 06 20 …	2: byte string, length in the next 2 bytes	1,568 bytes follow
A1 04 41 32	5: map with 1 pair	{4: h'32'} — a COSE key id (kid)

Deterministic encoding (RFC 9528 §3.1): always the shortest form, so `19 00 03` for 3 is rejected.

4. Inside the Messages (2/8) message_1, byte by byte

```
Test vector (fixed) – RFC 9529 trace 2: message_1, 39 bytes [offsets]
[0]      03                              METHOD = 3 (static DH / static DH)
[1-3]    82 06 02                        SUITES_I = [6, 2]: prefers 6, selected 2 (the last)
[4-5]    58 20                           CBOR head: byte string of 32
[6-37]   8a f6 f4 30 ... 2f c3 b6        G_X: x-coordinate of the P-256 key
[38]     37                              C_I = -24, a one-byte connection identifier
```

- Only the x-coordinate of G_X is sent; the receiver checks that it lies on the curve (invalid trace 4.2.3).
- The first message_1 of trace 2 offered suite 6 only. The responder answered with an **error message** `02 02` (ERR_CODE 2 = wrong selected cipher suite, SUITES_R = 2), and the initiator retried with [6, 2].
- Our boards offer one suite, sent as a plain integer: `03 02 58 20 … 2D` (37 bytes, C_I = −14). Between runs only G_X changes (a fresh key every run).

4. Inside the Messages (3/8) message_2: one byte string, partly encrypted

```
RFC 9529 trace 2: message_2, 45 bytes
58 2b                                    byte string of 43 bytes:
   41 97 01 d7 ... 3a 4f f5 d5           G_Y (32 B)
   98 62 a1 ee f9 e0 e7 e1 88 6f cd      CIPHERTEXT_2 (11 B)
```

```
PLAINTEXT_2 = CIPHERTEXT_2 XOR KEYSTREAM_2   (11 bytes)
27                                       C_R = -8
32                                       ID_CRED_R, compact: kid 0x32, i.e. {4: h'32'}
48 09 43 30 5c 89 9f 5c 54               Signature_or_MAC_2 = MAC_2 (byte string of 8)
```

- KEYSTREAM_2 = EDHOC_KDF(PRK_2e, 0, TH_2, 11). A plain XOR, no AEAD tag: message_2 is authenticated by the MAC_2 or signature inside it.
- A one-byte kid that is also a valid one-byte CBOR integer travels as that integer (0x32 reads as −19); the receiver turns it back into {4: h'32'}.
- Method 3, suite 2: the MAC is the suite's MAC length, 8 bytes. On your board G_Y, CIPHERTEXT_2 and MAC_2 change every run; C_R and the kid do not.

4. Inside the Messages (4/8) EDHOC_KDF: one function for every key

```
EDHOC_Extract(salt, IKM)         = HMAC-SHA-256(key = salt, IKM)        (HKDF-Extract)
EDHOC_KDF(PRK, label, ctx, L)    = HKDF-Expand(PRK, info, L)
    info = CBOR sequence ( label : int, context : bstr, length : uint )
```

```
board A, suite 2, DEBUG=1 (annotated)
info (size 36):                          -> KEYSTREAM_2
  00                                     label 0
  58 20 13 FB D3 29 ... 84 62 69 F3      context = TH_2 (32 B)
  0B                                     length 11
info (size 37):                          -> SALT_3e2m
  01                                     label 1
  58 20 13 FB D3 29 ... 84 62 69 F3      context = TH_2
  18 20                                  length 32
```

- Label and context keep every derived key apart; the length is an input too.
- PRK_2e = Extract(salt = TH_2, IKM = G_XY).
- SHA-384 in suite 24. Suite 25 (SHAKE256): Extract and Expand are KMAC256.

4. Inside the Messages (5/8) Signature_or_MAC_2: a MAC or a signature

```
context_2 = << C_R, ID_CRED_R, TH_2, CRED_R, ? EAD_2 >>
MAC_2     = EDHOC_KDF( PRK_3e2m, 2, context_2, mac_length_2 )
```

Responder proves itself by	mac_length_2	Signature_or_MAC_2
static DH (methods 1, 3)	the suite's MAC length: 8 (suites 0, 2) or 16	MAC_2 itself
signature (methods 0, 2)	the hash length (32 or 48)	signature over ["Signature1", << ID_CRED_R >>, << TH_2, CRED_R, ? EAD_2 >>, MAC_2]

```
board A, DEBUG=1:  C_R | ID_CRED_R | TH_2 | CRED_R
MAC context (size 94):
  0E  A1 04 41 02  58 20 13 FB ... 69 F3  A2 02 64 52 65 73 70 08 A1 01 A5 ...
```

- Static DH: PRK_3e2m mixes in G_RX, which only the initiator (ephemeral x) and the owner of the static key r can compute. A correct MAC proves the identity.
- CRED_R here is a CCS, {2: "Resp", 8: {1: COSE_Key}}; the key's y-coordinate is sent as the boolean false (`22 F4`).

4. Inside the Messages (6/8) message_3: a real AEAD

```
TH_3        = H( TH_2, PLAINTEXT_2, CRED_R )
K_3, IV_3   = EDHOC_KDF( PRK_3e2m, 3 | 4, TH_3, key_length | iv_length )
A_3         = [ "Encrypt0", h'', TH_3 ]        COSE Enc_structure, the AAD (45 B)
PLAINTEXT_3 = ( ID_CRED_I, Signature_or_MAC_3, ? EAD_3 )
message_3   = bstr( AEAD(K_3, IV_3, A_3, PLAINTEXT_3) )
```

```
RFC 9529 trace 2: message_3, 19 bytes
52                                      byte string of 18 bytes:
   e5 62 09 7b c4 17 dd 59 19 48        PLAINTEXT_3 encrypted: kid 0x2b, MAC_3 (bstr of 8)
   5a c7 89 1f fd 90 a9 fc              AES-CCM-16-64-128 tag (8 B)
```

- MAC_3 = EDHOC_KDF(PRK_4e3m, 6, << ID_CRED_I, TH_3, CRED_I, ? EAD_3 >>, …); PRK_4e3m mixes in G_IY when the initiator uses static DH.
- TH_4 = H(TH_3, PLAINTEXT_3, CRED_I) and PRK_out = EDHOC_KDF(PRK_4e3m, 7, TH_4, hash_length): the handshake is over.

4. Inside the Messages (7/8) Check it yourself from a DEBUG=1 console

```
board A, suite 2 method 3, DEBUG=1 (shortened)
Input to calculate TH_2 (CBOR Sequence) (size 68):
  58 20 E9 D8 46 C0 ... 4A AA 24 32  58 20 BD 3E 6C C6 ... EF DE EA 8E
TH2 (size 32):
  13 FB D3 29 B8 55 F8 BB D4 18 3F 2C E0 92 DE 0C ...
ciphertext2 (size 11):   16 83 69 B5 ED EC 30 2B DB 8C C8
KEYSTREAM_2 (size 11):   18 81 21 6B 7D 9D E2 77 13 6B F8
plaintext (size 11):     0E 02 48 DE 90 71 D2 5C C8 E7 30
```

```
Run this — from fpga/sw/export/edhoc-wolfssl; prints the TH2 of the log
make edhoc_m3_initiator edhoc_m3_responder CRYPTO_SUITE=2 DEBUG=1
../common/board/run_pair.sh s2_m3 build/edhoc_m3_initiator.elf build/edhoc_m3_responder.elf
awk '/Input to calculate TH_2/{f=1;next} f&&/^ *[0-9A-F][0-9A-F] /{printf "%s",$0;next} f{exit}' \
    boardlogs/s2_m3.ini.log | tr -d ' ' | python3 -c \
    "import sys,hashlib; print(hashlib.sha256(bytes.fromhex(sys.stdin.read())).hexdigest())"
```

- TH_2 = SHA-256 of (G_Y as bstr, H(message_1) as bstr): the 68 bytes above.
- ciphertext2 XOR KEYSTREAM_2, byte by byte, gives the plaintext: C_R = 0x0E, kid 0x02, then the 8-byte MAC_2.
- Your bytes differ every run (fresh ephemeral keys); these relations hold every time.

4. Inside the Messages (8/8) Methods 0, 1, 2: what changes in this walkthrough

	Responder proof in message_2	**PRK_3e2m**	**Initiator proof in message_3**	**PRK_4e3m**
Method 3 (this example)	MAC_2, 8 B	mixes G_RX	MAC_3, 8 B	mixes G_IY
Method 2	signature (ES256: 64 B)	= PRK_2e	MAC_3, 8 B	mixes G_IY
Method 1	MAC_2, 8 B	mixes G_RX	signature (64 B)	= PRK_3e2m
Method 0	signature (64 B)	= PRK_2e	signature (64 B)	= PRK_3e2m

- The message layout and the transcript hashes stay the same; only Signature_or_MAC and the static-DH mixing change.
- A signature side needs a signature key and the peer needs its public key; a static-DH side needs a DH key of the suite's curve.
- Try it: build method 0 with DEBUG=1 and find where “MAC 2/3 (size 32)” and the signature appear instead of the 8-byte MAC.

Outline

1. EDHOC in a Nutshell
2. Run One Handshake
3. Key Schedule
4. Inside the Messages
5. **The Library**
6. What to Expect
7. Practice

5. The Library (1/5) Where the code comes from

Part	Source	Folder
EDHOC protocol	uoscore-uedhoc (Fraunhofer AISEC, Apache-2.0 / MIT), EDHOC part only, heavily reworked	edhoc-wolfssl/uoscore-uedhoc
EDHOC core shared with Lesson 4	messages, CBOR, error messages, EAD	common/edhoc, common/cbor
SHA-2, HMAC/HKDF, AES-CCM/GCM	our own	common/crypto
ChaCha20/Poly1305, Ed25519	Monocypher	common/externals/monocypher
X25519	curve25519-donna (64-bit)	common/externals/curve25519-donna
P-256 / P-384, X448, Ed448, SHAKE256	wolfCrypt 5.9.1 subset (GPLv2)	edhoc-wolfssl/wolfssl

Each primitive comes from the smallest library that has it; only the files a suite needs are compiled.

5. The Library (2/5) The application: edhoc_initiator.c

edhoc-wolfssl/src/edhoc_initiator.c (shortened)

```
int main(void)
{
    board_init(0);
    csprng_add_entropy(role, sizeof(role));
    app_ephemeral_key(eph_sk, eph_pk);              // "PERF keygen"
    app_initiator_ctx(&ctx, &peer, eph_sk, eph_pk); // method, suites, keys, ID_CRED

    enum err r = edhoc_initiator_run(&ctx, &peers, &err_msg, &prk_out,
                                     tx_initiator, rx_initiator, ead_process);
    if (r != ok) { kprintf("EDHOC FAIL: %d\r\n", r); while (1); }
    kprintf("EDHOC OK!\r\n");
    board_report(t0, t1);                           // PERF + STACK lines
    board_app_data(&prk_out, 1);                    // exporter -> REQ/ECHO -> file
    edhoc_wipe(prk_out_buf, sizeof(prk_out_buf));
}
```

- One call runs the whole handshake; *tx* / *rx* are the transport (radio or UART).
- *peers* = trust anchors: the credentials we accept.
- The responder is the mirror image: *edhoc_responder_run()*.

5. The Library (3/5) Made strict to RFC 9528

Rule (RFC 9528)	Before	Now
Deterministic CBOR (§3.1)	accepted e.g. 19 00 03 for 3	rejected
SUITES_I: int or array of ≥ 2	array of 1 accepted	rejected
Compact C_x / kid (§3.3.2, §3.5.3.2)	loose	int form required
P-256/P-384: 0 ≤ x < p (§9.2)	x reduced mod p silently	rejected
kid is a byte string (COSE)	{4: 1}	{4: h'01'}
Unknown critical EAD (§3.8)	ignored	error, handshake stops
Static DH key with y = false	≈ 50 % had odd y	always even y
ID_CRED to the application (§3.5)	not exposed	id_cred_check() hook

Found by running the RFC 9529 vectors and reading every MUST; all fixed. Builds from before these fixes do not interoperate with builds after them.

5. The Library (4/5) Checking against RFC 9529

```
cd edhoc-wolfssl && make rfc9529-test   (excerpt)
Trace 2 (RFC 9529 Section 3): METHOD 3, suite 2, kid
  initiator message_1: PASS
  initiator message_3: PASS
  initiator PRK_out: PASS
  initiator PRK_exporter: PASS
  initiator OSCORE master secret: PASS
  initiator ID_CRED_R to application: PASS
  responder message_2: PASS
  responder PRK_out: PASS
  invalid 4.1.1 message_1 as array rejected: PASS (err 120, want 120)
  invalid 4.2.2 G_X x >= p rejected: PASS (err 4, want 4)
  invalid 4.2.3 G_X not on P-256 rejected: PASS (err 3, want 3)
  invalid 4.2.5 MAC_2 too short rejected: PASS (err 113, want 113)
  invalid RFC 9528 3.8 critical EAD_1 rejected: PASS (err 124, want 124)
RFC9529 TEST suite 2: PASS (0 failures)
```

- RFC 9529 publishes two complete example handshakes with every key and message.
- Our library, given the same keys and random values, must produce them **byte for byte**, in both roles.
- Plus the RFC's 15 invalid messages: each must be rejected with the right error.
- Scope: the RFC has traces for **two** combinations only (method 0 suite 0, method 3 suite 2); the suite 24 build runs the P-384 invalid case. The other 34 combinations have no published reference: they are checked by both sides agreeing (loopback, boards).

5. The Library (5/5) Demo profile vs. production profile

	PROFILE=demo (default)	**PROFILE=production**
Long-term keys	fixed test keys in the source	PROVISION_SRC: your key storage
Random numbers	SHA-256 CSPRNG seeded from the cycle counter	edhoc_platform_entropy(): a real TRNG
Use	lab, teaching, measurements	anything that matters

- RFC 9528 §9.2: ephemeral keys need good randomness. The demo seed is **not** good enough for real use — the production build refuses to build without a real entropy source.
- This FPGA SoC has no TRNG, so here only PROFILE=demo runs. To use the code for real, connect *edhoc_platform_entropy()* to a hardware entropy source.

Outline

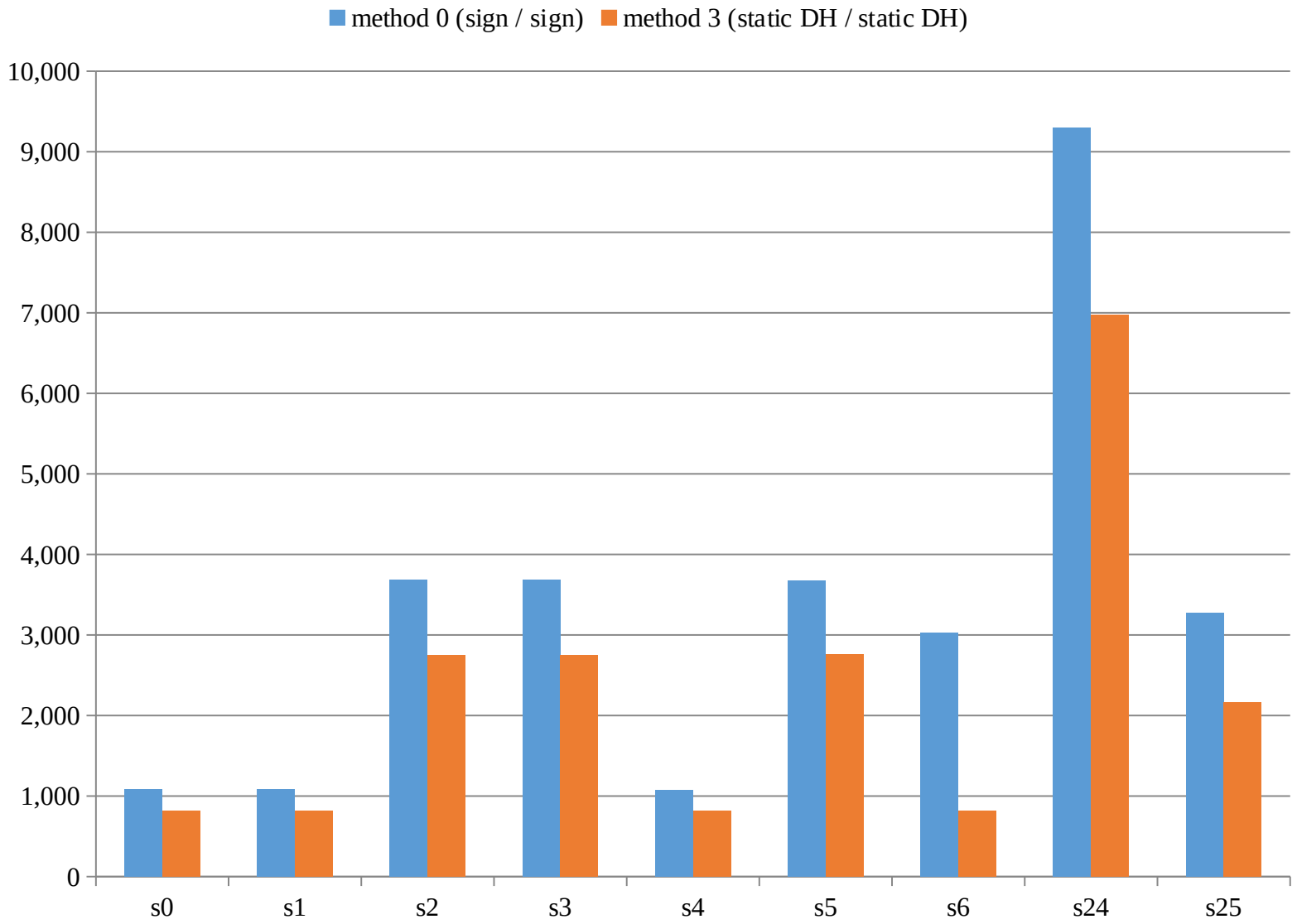
1. EDHOC in a Nutshell
2. Run One Handshake
3. Key Schedule
4. Inside the Messages
5. The Library
6. **What to Expect**
7. Practice

6. What to Expect (1/7) How these times were measured

Setting	Value used for every table in this section
Firmware	this revision, `TIMING=1`, **no** DEBUG=1 (printing would be timed too)
Clock	75 MHz; seconds = cycles / 75,000,000
Transport	MRF24J40 radio, channel 11 (TRANSPORT=rf, the default)
Role	initiator, unless the column says R
handshake_total	from the call that builds message_1 until the handshake returns; excludes ephemeral key generation (PERF keygen)
handshake_compute	total minus the time spent waiting for the link
message steps	compute of one step only (build or process one message)

- The responder's total also counts waiting for the initiator to be loaded: compare compute times between the two sides, not totals.
- Sender and receiver steps overlap in time — do not add them up to get a handshake time. Debugging (DEBUG=1) and benchmarking (TIMING=1) are separate builds.

6. What to Expect (2/7) Handshake time over the radio



- Whole handshake at the initiator, in **ms**, over the radio.
- Every pair should end in PASS with each stack inside its reserve — run_pair.sh checks both.
- Fastest: X25519 suites with method 3, **0.81 s**. Slowest: suite 24 (P-384) with method 0, **9.3 s**.
- Method 3 is faster than method 0 for every suite: a MAC is cheaper than a signature.

6. What to Expect (3/7) Time of every message step — method 3 (ms)

Suite	I: build m1	R: process m1 + build m2	I: process m2	I: build m3	R: process m3	I: compute
0 / 1	0.5	251.6	374.6	10.7	133.5	390.6
2 / 3	0.5	892.0	1352.6	10.4	454.1	1368.0
4	0.5	254.3	374.2	10.2	135.1	389.5
5	0.5	903.1	1336.9	10.6	457.2	1352.8
6	0.5	254.1	374.4	10.8	135.4	390.6
24	1.2	2287.6	3446.2	23.2	1160.9	3480.8
25	0.5	703.5	1052.6	8.4	358.0	1065.2

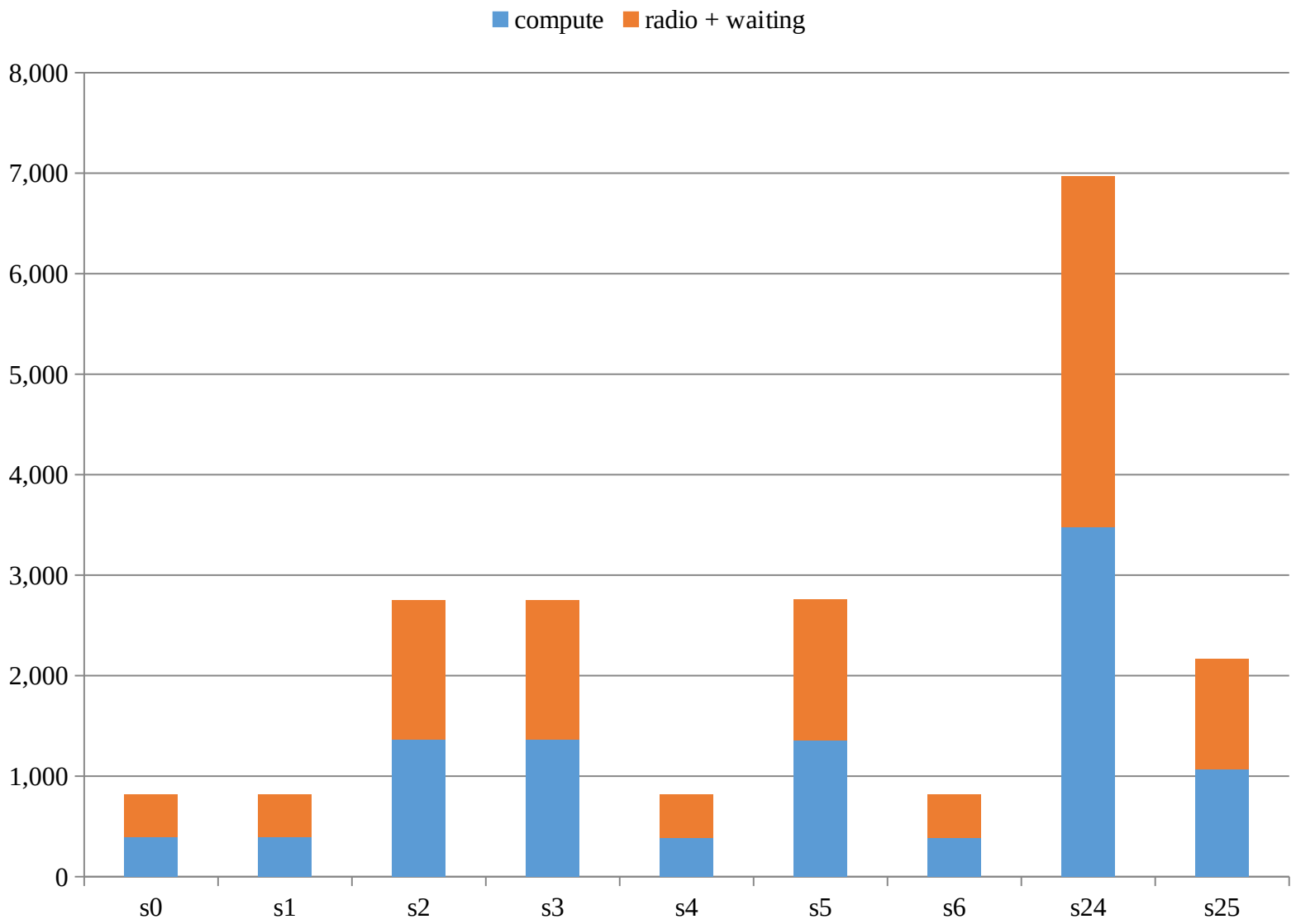
- I = initiator (board A), R = responder (board B); `TIMING=1` builds.
- Building message_1 is almost free; the key exchanges inside *process m2* dominate.

6. What to Expect (4/7) Time of every message step — method 0 (ms)

Suite	I: build m1	R: process m1 + build m2	I: process m2	I: build m3	R: process m3	I: compute
0 / 1	0.5	219.3	422.8 / 418.1	96.1	297.2	524.2
2 / 3	0.5	931.1	1331.9	486.6	885.9	1823.9
4	0.5	218.5	416.6	95.5	292.7	517.2
5	0.5	931.5	1329.8	485.3	885.6	1820.1
6	0.5	607.0	1011.9	484.2	883.1	1501.5
24	1.2	2378.2	3369.5	1242.5	2234.7	4623.4
25	0.5	1136.9	814.6	789.9	468.1	1608.7

- With signatures, *build m3* (sign) and *process m3* (verify) become expensive.
- Suite 6 mixes X25519 key exchange with ES256 (P-256) signatures — fast DH, slow signatures.

6. What to Expect (5/7) Where the time goes



- Method 3, initiator, ms.
- Compute is the elliptic-curve work: P-256 ≈ 3.5× and P-384 ≈ 9× the cost of X25519 on this core.
- The rest is the radio and the peer's compute: messages cross a 250 kbit/s link in ~100-byte fragments.
- Software only: this core's multiplier is slow (~64 cycles), so the curve code uses 64-bit limbs to need as few multiplies as possible.

6. What to Expect (6/7) Memory

Suite	Largest image (B)	Stack reserve (B)	Total (B)	of 65,536
0 / 1	42,291	5,376	47,667	73 %
2 / 3	43,739	8,448	52,187	80 %
4	41,653	5,376	47,029	72 %
5	43,365	8,448	51,813	79 %
6	46,361	8,448	54,809	84 %
24	47,391	11,008	58,399	89 %
25	47,913	12,544	60,457	92 %

- Largest of the 8 builds per suite (4 methods × 2 roles). Stack reserve = measured peak + 25 %.
- Every build links (image + reserve fit) and every board run stayed inside its reserve (STACK line) — Lesson 2 shows both checks.

6. What to Expect (7/7) What you should see

Run	You should see
Any method × suite on two boards (run_pair.sh)	PASS, stacks inside their reserve
Host loopback, same combination	PRK_out match -> PASS
RFC 9529 test (2 traces + 15 invalid messages)	PASS in builds of suites 0, 2, 24
Handshake time at the initiator	0.81 s (X25519, method 3) … 9.3 s (P-384, method 0)
Largest image + stack	60,457 B of 65,536 B (suite 25)
Before real use	a real entropy source (TRNG) instead of the demo seed

Outline

1. EDHOC in a Nutshell
2. Run One Handshake
3. Key Schedule
4. Inside the Messages
5. The Library
6. What to Expect
7. **Practice**

7. Practice (1/3) Exercises

Exercise 1:

- Run suite 0 method 3 and suite 24 method 0 on your boards with `TIMING=1`. Compare each message step with the tables.

Exercise 2:

- Run `make rfc9529-test`. Change one byte of a test vector in *tests/rfc9529_vectors.h*: which checks fail?

Exercise 3:

- Give the initiator an `id_cred_check()` that refuses the responder's kid 0x02. What does each console print, and why does the initiator stop before message_3?

7. Practice (2/3) What to put in the report

- Exercise 1: both console logs per run and a table of your step times next to ours; explain any difference.
- Exercise 2: the failing lines and which RFC value they compare.
- Exercise 3: your hook, both console logs, and the RFC 9528 section that requires the hook.

7. **Practice** (3/3) Answer notes

Exercise	What you should see	Why
1. Step times	within a few % of the tables; compute stable run to run, totals vary with the radio	compute is deterministic; the link is not
2. Change an input (X, Y, SK_x)	almost every later check FAILs: messages, PRK_out, exporter	each key depends on the transcript
2. Change an expected output	only that one check FAILs	the computed value did not change
3. Refuse kid 0x02	initiator: EDHOC FAIL after message_2, no message_3; responder: EDHOC FAIL (error message received); no APP lines	ID_CRED_R arrives in PLAINTEXT_2 — the check runs before message_3 is built

- Negative case to think through: if the **responder** refuses ID_CRED_I instead, the initiator has already sent message_3 and printed EDHOC OK — only the application step fails. One EDHOC OK is not enough; read both consoles.
- Restore: remove the hook (`id_cred_check = NULL`), rebuild both roles, rerun: PASS.

Today's lesson highlight:

- EDHOC = 3 messages → mutual authentication + PRK_out; 4 methods (signature or static DH per side), 9 cipher suites.
- Every key is bound to the transcript (TH_2, TH_3, TH_4); applications take their keys with EDHOC_Exporter.
- Library made strict to RFC 9528 and checked byte for byte against RFC 9529.
- Every method and suite runs on two boards over the radio: 0.81 s … 9.3 s; TIMING=1 shows the time of every message step.
- Read results as a ladder: EDHOC OK (one side) → APP DATA OK (compatible keys) → prk=match (equal PRK_out) → RFC 9529 (equal to the standard).

Next lesson:

「 Lesson 4 」 Post-quantum EDHOC

THANK YOU

[YYYY/MM]